Historical data analysis

Objective

Analyze historical stock price data and:

- Understand **time-series data**
- Calculate **rolling averages (moving averages)**
- Visualize **price trends**
- Identify basic buy/sell signals

```
!pip install yfinance
import yfinance as yf

ticker = "AAPL"                                          # stock symbol you want
start_date = "2016-01-01"
end_date = "2025-12-31"

data = yf.download(ticker, start=start_date, end=end_date)
data.to_csv(f"{ticker}_historical.csv")
print("Saved:", f"{ticker}_historical.csv")
```

```
[*********************100%***********************]  1 of 1 completed
Saved: AAPL_historical.csv
```

```
import pandas as pd
data = pd.read_csv("AAPL_historical.csv")
data
```

	Price	Close	High	Low	Open	Volume
0	Ticker	AAPL	AAPL	AAPL	AAPL	AAPL
1	Date	NaN	NaN	NaN	NaN	NaN
2	2016-01-04	23.730947494506836	23.735453620243067	22.976332980529655	23.113740599566555	270597600
3	2016-01-05	23.136266708374023	23.843577233061005	23.068690287724074	23.821051759511022	223164000
4	2016-01-06	22.683490753173828	23.059673099076566	22.49652782994531	22.651954755589113	273829600
...	...	...	...	...	...	...
2510	2025-12-23	272.1053771972656	272.2452609569047	269.30800689497966	270.5868091030317	29642000
2511	2025-12-24	273.55401611328125	275.17249671786806	271.9455359163183	272.085389177814	17910600
2512	2025-12-26	273.1444091796875	275.1125687682936	272.6049054553	273.9037084592662	21521800
2513	2025-12-29	273.50408935546875	274.10350406329195	272.0954038139857	272.4350823114352	23715200
2514	2025-12-30	272.82470703125	273.82377221524894	272.02546707967076	272.5549704076282	22139600

2515 rows × 6 columns

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2515 entries, 0 to 2514
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Price   2515 non-null   str
 1   Close   2514 non-null   str
 2   High    2514 non-null   str
```

```
 3   Low       2514   non-null   str
 4   Open      2514   non-null   str
 5   Volume    2514   non-null   str
dtypes: str(6)
memory usage: 118.0 KB
```

Note:-

What's Wrong in Your Data?

Your DataFrame:
Columns:
0 Price (str)
1 Close (str)
2 High (str)
3 Low (str)
4 Open (str)
5 Volume (str)

Problems:

1. ❌ No proper **Date column**
2. ❌ All columns are **string (str)** type
3. ❌ "Price" column should probably be the **Date**
4. ❌ Numeric columns are stored as text
5. ❌ One missing row (2515 vs 2514)

Step 1 — Check First Few Rows

First run:

```
df.head()
```

	Price	Close	High	Low	Open	Volume
0	Ticker	AAPL	AAPL	AAPL	AAPL	AAPL
1	Date	NaN	NaN	NaN	NaN	NaN
2	2016-01-04	23.730947494506836	23.735453620243067	22.976332980529655	23.113740599566555	270597600
3	2016-01-05	23.136266708374023	23.843577233061005	23.068690287724074	23.821051759511022	223164000
4	2016-01-06	22.683490753173828	23.059673099076566	22.49652782994531	22.651954755589113	273829600

Now,

- Your dataset has **extra header rows** created by yfinance multi-level columns.
- Your first 2 rows are not real data:
- Row 0 → Ticker AAPL AAPL AAPL AAPL AAPL
- Row 1 → Date NaN NaN NaN NaN NaN
- Row 2 → Actual data starts here

So we just need to clean this properly.

Clean Your Exact Dataset (Step-by-Step)

Step 1 — Skip First Two Rows

```
df = pd.read_csv("AAPL_historical.csv", skiprows=2)
```

OR if already loaded:

```
df = df.iloc[2:]
df
```

	Date	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5
0	2016-01-04	23.730947	23.735454	22.976333	23.113741	270597600
1	2016-01-05	23.136267	23.843577	23.068690	23.821052	223164000
2	2016-01-06	22.683491	23.059673	22.496528	22.651955	273829600
3	2016-01-07	21.726145	22.555095	21.721640	22.228471	324377600
4	2016-01-08	21.841036	22.325341	21.795985	22.199197	283192000
...	...	...	...	...	...	...
2508	2025-12-23	272.105377	272.245261	269.308007	270.586809	29642000
2509	2025-12-24	273.554016	275.172497	271.945536	272.085389	17910600
2510	2025-12-26	273.144409	275.112569	272.604905	273.903708	21521800
2511	2025-12-29	273.504089	274.103504	272.095404	272.435082	23715200
2512	2025-12-30	272.824707	273.823772	272.025467	272.554970	22139600

2513 rows × 6 columns

Step- 2

This is the **last cleaning step**.
Your data is now almost correct — only column names are wrong.

Right now you have:
Date | Unnamed:1 | Unnamed:2 | Unnamed:3 | Unnamed:4 | Unnamed:5

But the values clearly represent:
Date | Close | High | Low | Open | Volume

So we just need to rename columns properly.

```
df.columns = ['Date', 'Close', 'High', 'Low', 'Open', 'Volume']
df
```

	Date	Close	High	Low	Open	Volume
0	2016-01-04	23.730947	23.735454	22.976333	23.113741	270597600
1	2016-01-05	23.136267	23.843577	23.068690	23.821052	223164000
2	2016-01-06	22.683491	23.059673	22.496528	22.651955	273829600
3	2016-01-07	21.726145	22.555095	21.721640	22.228471	324377600
4	2016-01-08	21.841036	22.325341	21.795985	22.199197	283192000
...	...	...	...	...	...	...
2508	2025-12-23	272.105377	272.245261	269.308007	270.586809	29642000
2509	2025-12-24	273.554016	275.172497	271.945536	272.085389	17910600
2510	2025-12-26	273.144409	275.112569	272.604905	273.903708	21521800
2511	2025-12-29	273.504089	274.103504	272.095404	272.435082	23715200
2512	2025-12-30	272.824707	273.823772	272.025467	272.554970	22139600

2513 rows × 6 columns

Step 3: Convert Date to Datetime

```
df['Date'] = pd.to_datetime(df['Date'])
```

Step 4 — Set Date as Index (VERY IMPORTANT for time series)

```
df.set_index('Date', inplace=True)
df.sort_index(inplace=True)
```

Why this is important?

- pd.to_datetime() → converts string to proper date format
- set_index('Date') → makes it time-series data
- Now pandas understands time-based operations

Notes:-

1. What Happens Before Setting Date as Index?

```
df.head()
```

Output

	Date	Close
0	2016-01-04	23.73
1	2016-01-05	23.13
2	2016-01-06	22.68

Here:

- Date is just a **normal column**
- Index is just 0,1,2,3...

So pandas treats this as **regular tabular data**, not time-series.

2. After set_index('Date')

```
df.set_index('Date', inplace=True)
```

Output:

Date	
2016-01-04	23.73
2016-01-05	23.13
2016-01-06	22.68

Now the index becomes: ***DatetimeIndex***

Now pandas knows:

- 👉 This data is ordered by time
- 👉 Each row represents a specific timestamp
- 👉 Time-based slicing is possible

Why This Is IMPORTANT

✅ **1. Time-Based Filtering Becomes Easy**

Without index:

```
df[df['Date'] >= '2020-01-01']
```

With index:

```
df.loc['2020']
```

Or even:

```
df.loc['2020-01':'2020-06']
```

That's powerful.

✅ **2. Rolling Functions Work Properly**

When index is datetime, pandas understands sequence over time.

Example:

```
df['Close'].rolling(20).mean()
```

It calculates based on **chronological order**.

If data is not sorted by date → wrong results.

✅ 3. Resampling Becomes Possible

This is HUGE in time-series.

Example:
Daily → Monthly average

```
df.resample('M').mean()
```

This only works if:

- Date is datetime
- Date is index

✅ 4. Time-Based Grouping

Example:

```
df['Close'].groupby(df.index.year).mean()
```

You can group by:

- Year
- Month
- Quarter
- Week

Only possible with DatetimeIndex.

✅ 5. Automatic Plotting Over Time

When plotting:

```
df['Close'].plot()
```

Matplotlib automatically:

- Uses dates on X-axis
- Formats time properly

🧠 Interview-Level Explanation

Q: Why is setting Date as index important in time-series analysis?

Answer:

> Setting the Date column as a DatetimeIndex allows pandas to recognize the dataset as time-series data. It enables time-based slicing, resampling, rolling calculations, chronological sorting, and efficient trend analysis.

That answer shows deep understanding 👌

🎯 Real Example From Your Stock Project

When analyzing stock of
Apple Inc.

If Date is not index:

- You cannot easily compute yearly returns
- You cannot resample to monthly average
- Time-based slicing becomes complicated

So in financial analysis → **Date as index is mandatory best practice**

Step 5 — Verify

```
df.info()
df.head()
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 2513 entries, 2016-01-04 to 2025-12-30
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Close   2513 non-null   float64
 1   High    2513 non-null   float64
 2   Low     2513 non-null   float64
 3   Open    2513 non-null   float64
 4   Volume  2513 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 117.8 KB
```

Date	Close	High	Low	Open	Volume
2016-01-04	23.730947	23.735454	22.976333	23.113741	270597600
2016-01-05	23.136267	23.843577	23.068690	23.821052	223164000
2016-01-06	22.683491	23.059673	22.496528	22.651955	273829600
2016-01-07	21.726145	22.555095	21.721640	22.228471	324377600
2016-01-08	21.841036	22.325341	21.795985	22.199197	283192000

✅ Clean
✅ Structured
✅ Time-series ready

Why This Happened?

This happens when downloading data using:
`yf.download("AAPL")`
from Apple Inc. via the yfinance library.
By default, it creates **multi-index column format** which gets saved strangely in CSV

STEP 6: Basic Line Plot (Trend Visualization)

```
plt.figure(figsize=(10,5))
plt.plot(df['Close'])
plt.title("Closing Price Trend")
plt.xlabel("Date")
plt.ylabel("Price")
plt.show()
```

📊 **What you observe:**

📈 1️⃣ **Is the Stock Going Upward?**

Yes — strong long-term upward trend

Observations:

- 2016 price ≈ 20–25
- 2026 price ≈ 270–280
- Clear higher highs over time

This indicates:

- Strong long-term growth
- Bullish overall trend
- Company value appreciation

📌 ***Interview way to say it:***

The stock shows a clear long-term upward trend with consistent higher highs, indicating sustained growth over the observed period.

🌊 2️⃣ **Is It Volatile?**

Yes — especially after 2020

Notice:

- 2016–2018 → relatively smooth growth
- 2020 onward → larger swings
- Bigger up and down movements

Volatility increased after 2020.

That period aligns with:

- COVID-19 pandemic market crash and recovery

📌 ***Interview answer:***

The stock demonstrates increasing volatility after 2020, with larger price fluctuations compared to earlier years.

💥 3️⃣ **Any Sudden Crash or Spike?**

🔻 **Major Crash Around 2020**

Sharp drop visible — likely pandemic-driven market crash.

🔺 **Strong Recovery After 2020**

Very aggressive upward rally.

🔻 **Corrections Around 2022**

You can see noticeable dips before continuing upward.

📌 ***Interview-level interpretation:***

The chart shows a significant correction around early 2020 followed by a strong recovery. Additional pullbacks appear around 2022, indicating cyclical corrections within an overall bullish trend.

STEP 5: Rolling Averages (Moving Averages)

Moving average smooths data and shows trend clearly.

1. 📌 **20-Day Moving Average**

```
df['MA20'] = df['Close'].rolling(window=20).mean()
df['MA20']
```

Output:- Date

```
2016-01-04        NaN
2016-01-05        NaN
2016-01-06        NaN
              ...
2025-12-23  277.050758
2025-12-24  276.892905
2025-12-30  275.930804
Name: MA20, Length: 2513, dtype: float64
```

Why Are First 19 Values NaN?

You used: *'df['MA20'] = df['Close'].rolling(window=20).mean()'*

This means:

👉 For each row, pandas needs **20 previous values** to compute the average.

For example: To calculate MA20 on: *2016-01-23*

Pandas needs data from: *2016-01-04 to 2016-01-23 (20 days)*

❗ **For first 19 rows:**

There are NOT enough previous values.

So pandas returns: *NaN (Not a Number)*

Simple Example

Imagine Close prices: 10, 20, 30, 40
If window=3:

Day	Rolling Mean
10	NaN
20	NaN
30	20
40	30

Because:
- At 30 → (10+20+30)/3
- At 40 → (20+30+40)/3

🔷 **When Does MA20 Start Showing Values?**

It starts at the **20th row**.

So:
First 19 → NaN
From row 20 → valid values

This is mathematically correct.

Can We Remove NaN?

If needed: df.dropna(inplace=True)

⚠ But be careful:
- Dropping rows removes early historical data.
- Usually for plotting → no need to drop.

Want MA to Start Earlier?

You can use:

df['MA20'] = df['Close'].rolling(window=20, min_periods=1).mean()

Now it calculates with available data.

But ⚠ This is NOT true 20-day MA at beginning.

🔥 Interview-Level Explanation

Why do moving averages contain NaN at the beginning?

Answer:

Because the rolling window requires a minimum number of past observations equal to the window size. Until enough data points are available, pandas returns NaN.

That answer shows strong understanding 👌

🔷 Final Confirmation

Your last values like:

```
2025-12-23    277.05
2025-12-24    276.89
```

Mean:

👉 Average closing price of last 20 trading days ≈ 277

That shows smoothing effect.

---

2. 📌 50-Day Moving Average

```
df['MA50'] = df['Close'].rolling(window=50).mean()
df['MA50']
```

Output:- Date

```
2016-01-04        NaN
2016-01-05        NaN
2016-01-06        NaN
              ...
2025-12-29  271.717334
2025-12-30  272.137628
Name: MA50, Length: 2513, dtype: float64
```

Now for this:-

Let's understand clearly 👇

Why Are First 49 Values NaN?

Because:

👉 A 50-day moving average needs **50 previous closing prices**.
Until pandas collects 50 values, it cannot compute the average.

So:

- First 49 rows → ❌ Not enough data → NaN
- From 50th row onward → ✅ Proper MA50 values

Mathematical Explanation

If window = 50:

For date:
2016-03-15 (example)

MA50 = Average of:
Last 50 trading days' Close prices

If only 10 days exist → impossible to calculate true 50-day average.
So pandas returns NaN.

Why MA50 Starts Later Than MA20?

Because:

Indicator	Needs	NaN Count
MA20	20 values	19 NaN
MA50	50 values	49 NaN

- So MA50 line on graph starts later than MA20.
- This is completely normal in financial charts.

What Do Your Last Values Mean?

Example:

2025-12-30 272.13

This means:

- 👉 The average closing price of the last 50 trading days ≈ 272
- It smooths price more than MA20.

🔥 Why MA50 Is Smoother Than MA20?

Because:

- Larger window = More smoothing
- Less reaction to short-term noise
- Better for long-term trend

MA20 reacts faster
MA50 reacts slower

🔥 Interview-Level Explanation

If interviewer asks:

Why do longer moving averages have more lag?

Answer:

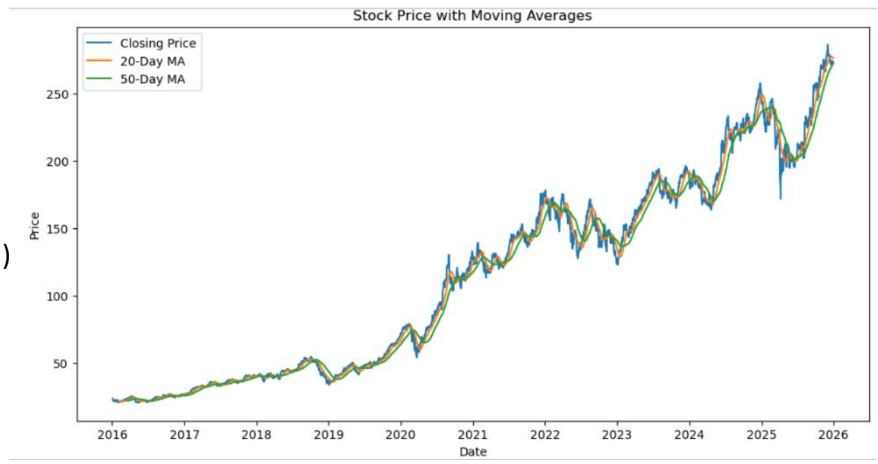
Because longer moving averages incorporate more historical data points, they respond more slowly to recent price changes, resulting in smoother but more delayed signals.

STEP 6: Plot Moving Averages

```
plt.figure(figsize=(12,6))

plt.plot(df['Close'], label='Closing Price')
plt.plot(df['MA20'], label='20-Day MA')
plt.plot(df['MA50'], label='50-Day MA')

plt.legend()
plt.title("Stock Price with Moving Averages")
plt.xlabel("Date")
plt.ylabel("Price")
plt.show()
```



Now let's analyze what we observe from your graph.

📈 1️⃣ Is Stock Going Upward?

✅ Yes — clearly upward long-term trend.

From 2016 to 2026:
- Price moves from around ~20
- To nearly ~280+

That shows strong long-term bullish trend.

Even though there are pullbacks, the overall direction is upward.
👉 So trend = Bullish

🌪 2️⃣ Is It Volatile?

✅ Yes — especially after 2020.

Notice:
- Bigger price swings
- Faster ups and downs
- Larger distance between highs and lows

Earlier (2016–2018) → smooth gradual rise
Later (2020 onward) → sharp movements

👉 Volatility increased over time.

⚡ 3️⃣ Any Sudden Crash or Spike?

Yes.

Around 2020:
- Sharp drop (likely market crash type movement)
- Then strong recovery

Also:
- 2022–2023 → noticeable corrections
- 2025 → visible sharp dip before rising again

So yes:
✓ Sudden dips
✓ Sudden strong rallies

📊 4️⃣ What Do Moving Averages Show?

🔶 **MA20 (Orange)**

- Reacts faster
- Moves closer to price
- Good for short-term signals

🟢 **MA50 (Green)**

- Smoother
- Lags more
- Shows long-term direction

🔥 Important Observations

When MA20 > MA50

Bullish momentum

When MA20 < MA50

Bearish momentum

You can see multiple crossover points in your chart.

Those are:

- Golden Cross (buy signal)
- Death Cross (sell signal)

🧠 Key Learning From This Chart

1. Long-term trend is upward
2. Volatility increased after 2020
3. Moving averages smooth noise
4. Crossovers indicate momentum change
5. MAs lag behind actual price

🎯 Interview-Level Answer (If Asked)

The stock shows a clear long-term bullish trend with increasing volatility after 2020. The 20-day moving average reacts quickly to price changes, while the 50-day moving average smooths long-term trends. Several crossover points indicate shifts in momentum.

🎯 What You Learn Here

Concept	What You Practiced
Time Series	Date indexing
Rolling Function	.rolling().mean()
Trend Analysis	Line plot
Data Smoothing	Moving average
Visualization	Matplotlib styling

1️⃣ Bearish Momentum (Death Cross) 🔴

Jab: **MA50 upar se neeche MA200 ko cross karta hai**

Matlab:

- Short-term trend weak ho gaya
- Recent prices gir rahe hain
- Long-term trend se neeche aa gaya

Isko kehte hain:

- 👉 **Death Cross**
- 👉 Bearish signal

Note:-
We can take MA60 and MA180 **OR** MA70 MA220 matlab kuch bhi le saktei hai, but recommended MA50 and MA200 hota hai

2️⃣ Bullish Momentum (Golden Cross) 🟢

Jab: **MA50 neeche se upar MA200 ko cross karta hai**

Matlab:

- Short-term strength aa rahi hai
- Recent prices strong ho rahe hain
- Long-term trend ko break kar diya

Isko kehte hain:

- 👉 **Golden Cross**
- 👉 Bullish signal

🎯 **So Tumhara Statement Tab Correct Hai Jab:**

Agar tum keh rahe ho: *"MA50 jab MA200 ko upar se neeche kaat deta hai toh bearish momentum hota hai"*

✓ Yes — bilkul correct.

Lekin agar sirf itna bolo:

"MA50 MA200 ko kaat deta hai toh bearish momentum"

❌ Incomplete statement hai — direction important hota hai.

🧠 **Interview Tip**

Why does this indicate bearish momentum?

Answer:
Because the short-term average falling below the long-term average suggests that recent prices are weakening relative to the long-term trend, indicating downward momentum.

Note:-

- **Chhota Moving Average (jaise MA50)** → Fast react karta hai
- **Bada Moving Average (jaise MA200)** → Slow react karta hai

---

IMPORTANT — Interview Insight

If interviewer asks:

Q1: "What challenges did you face in your stock data project?"

You can say:
The dataset had all columns stored as strings and the date column was mislabeled. I cleaned it by renaming columns, converting date to datetime format, removing commas from numeric values, converting them to numeric types, and setting date as index for proper time-series analysis.

That answer shows **real data handling skill** 💯

Q2: What data cleaning did you perform?

Say:
The downloaded dataset had extra header rows due to multi-index columns from yfinance. I removed unnecessary rows, renamed the date column properly, converted data types, handled missing values, and structured the dataframe as a proper time-series before analysis.

Q3: How did you clean the dataset?

You can confidently say:

The dataset initially had multi-level headers and unnamed columns due to export formatting. I renamed the columns properly, converted the Date column to datetime format, set it as index for time-series operations, and verified numeric types before performing rolling average analysis.

Note:-

🚀 **Alternative (Better Way – Cleaner CSV)**

Instead of saving directly, use:

```
data = yf.download("AAPL", start="2016-01-01", end="2025-01-01")
```

```
data.reset_index(inplace=True)
data.to_csv("AAPL_clean.csv", index=False)
```

This will save clean data like:

```
Date,Open,High,Low,Close,Adj Close,Volume
```

No extra rows.

Step 7: Now You Can Start Actual Analysis *i.e* Rolling Averages (Moving Averages)

Moving average smooths data and shows trend clearly.

📈 **Moving Averages**

```
df['MA20'] = df['Close'].rolling(20).mean()
df['MA50'] = df['Close'].rolling(50).mean()
```

📊 **Plot Trend**

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
plt.plot(df['Close'], label='Closing Price')
plt.plot(df['MA20'], label='20 Day MA')
plt.plot(df['MA50'], label='50 Day MA')
plt.legend()
plt.title("AAPL Stock Price Trend")
plt.xlabel("Date")
plt.ylabel("Price")
plt.show()
```

What You Learn Here

Concept	What You Practiced
Time Series	Date indexing
Rolling Function	`.rolling().mean()`
Trend Analysis	Line plot
Data Smoothing	Moving average
Visualization	Matplotlib styling

Interview Questions + Model Answers

Here are **common interview questions** you might face for a stock analysis role — with ready answers 👇

Q1 — What is a Time Series?

A: A *time series* is a series of data points indexed in chronological order, where each value represents the state of a variable (like stock price) at a specific time. Time series data allows us to analyze trends, seasonal patterns, and forecast future values.

Q2 — Why Set the Date as the Index?

A: Setting the 'Date' column as the index enables time-based slicing, rolling window functions (like moving averages), and efficient plotting. It makes the dataframe behave like actual time series data.

Q3 — What is a Rolling/Moving Average?

A: A rolling (or moving) average smooths out short-term fluctuations and highlights longer-term trends. For example, a 20-day moving average computes the average price of the past 20 trading days.

Q4 — Difference Between Simple MA and Exponential MA

A:

- *Simple Moving Average (SMA)* gives equal weight to all data points.
- *Exponential Moving Average (EMA)* gives more weight to recent prices, making it more responsive to new information.

Q5 — Why Use Adjusted Close Prices?

A: *Adjusted Close* reflects dividends and splits. It gives a more accurate picture of total return than the nominal closing price.

Q6 — What Can Moving Averages Tell Us?

A: They help identify:

- Trend direction (up, down, sideways),
- Crossovers (e.g., 20-day crossing above 50-day) that can indicate buy/sell signals,
- Smoothing noisy price movements.

Q7 — How Do Daily Returns Help?

A: *Daily returns* show percentage change day-to-day and help in:

- measuring volatility,
- comparing performance across stocks,
- risk analysis.

Q8 — How to Handle Missing Dates?

A: Financial time series may have missing dates due to weekends/holidays. We handle this by:

- forward-filling,
- backward-filling,
- or interpolation depending on analysis needs.

What is Time-Series Data?

✅ **Simple Definition:**
Time-series data is data collected **over time at regular intervals**.

📌 **Example:**
Stock price of Apple Inc. recorded daily from 2016–2025.

Date	Close
2016-01-04	23.73
2016-01-05	23.13
2016-01-06	22.68

🎯 **Key Characteristics:**

- Ordered chronologically
- Has time dependency
- Used for trend & forecasting analysis

🧠 **Interview-Ready Answer:**

Time-series data is sequential data indexed by time. It allows us to analyze trends, seasonality, patterns, and forecast future values. In stock market analysis, prices recorded daily form a time-series dataset.

2️⃣ Why Do We Use Moving Averages?

✅ **Purpose:**
To **smooth out short-term fluctuations** and highlight long-term trends.

Stock prices are noisy. Moving averages reduce noise.

📊 **Example:**

If prices jump: 20, 25, 18, 30, 27

Moving average smooths it to: 22, 24, 25...

🎯 **Why Important?**
- Identifies trend direction
- Detects buy/sell signals
- Reduces volatility impact

🧠 **Interview Answer:**

> Moving averages smooth short-term price fluctuations and help identify the underlying trend. They are widely used in technical analysis to detect trend direction and crossover signals.

3️⃣ What is a Rolling Window?

✅ **Definition:**

A rolling window calculates statistics over a **fixed number of previous observations**.

Example: `df['MA20'] = df['Close'].rolling(20).mean()`

This means:
- Take last 20 days
- Calculate average
- Move forward 1 day
- Repeat

🎯 **Why Used?**

To compute:
- Moving average
- Rolling standard deviation
- Rolling sum
- Rolling volatility

🧠 **Interview Answer:**

A rolling window is a technique where calculations are performed on a fixed-size subset of consecutive observations. It moves forward one step at a time and is commonly used in time-series analysis for smoothing and volatility measurement.

4️⃣ Difference Between Simple MA and Exponential MA

Feature	Simple MA (SMA)	Exponential MA (EMA)
Weight	Equal weight to all points	More weight to recent prices
Responsiveness	Slower	Faster
Noise Handling	Smooth	Reacts quickly
Usage	Long-term trend	Short-term trading

📌 **Formula Concept:**

- **SMA** = Average of last N values
- **EMA** = Weighted average (recent values matter more)

In pandas:

```
# Simple MA
df['SMA20'] = df['Close'].rolling(20).mean()

# Exponential MA
df['EMA20'] = df['Close'].ewm(span=20).mean()
```

🧠 **Interview Answer:**

The Simple Moving Average gives equal importance to all data points in the window, whereas the Exponential Moving Average assigns higher weight to recent prices, making it more responsive to new information.

5️⃣ **What is Volatility?**

✅ **Definition:**

Volatility measures how much the price **fluctuates over time**.

High volatility = large price swings
Low volatility = stable price movement

📊 **How To Calculate in Stock Analysis:**

```
df['Volatility'] = df['Close'].rolling(20).std()
```

(Standard deviation over 20 days)

📌 **Why Important?**

- Measures risk
- Important for trading decisions
- Used in portfolio management
- Used in options pricing models

🧠 **Interview Answer:**

Volatility measures the degree of variation in a stock's price over time, usually calculated using standard deviation of returns. Higher volatility indicates higher risk and larger price swings.

🎯 **BONUS: Real-World Application**

In stocks like:

- Apple Inc.
- Tesla, Inc.

Tesla historically shows higher volatility than Apple.

1️⃣ **Golden Cross Strategy**

✅ **What is Golden Cross?**

Golden Cross happens when: **Short-term Moving Average crosses ABOVE Long-term Moving Average**

Example: 50-Day MA crosses above 200-Day MA

It signals **bullish trend (buy signal)**.

Opposite is **Death Cross** (bearish signal).

📊 Code Implementation

```
df['MA50'] = df['Close'].rolling(50).mean()
df['MA200'] = df['Close'].rolling(200).mean()

df['Signal'] = 0
df.loc[df['MA50'] > df['MA200'], 'Signal'] = 1
```

📈 Plot Golden Cross

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
plt.plot(df['Close'], label='Close')
plt.plot(df['MA50'], label='MA50')
plt.plot(df['MA200'], label='MA200')
plt.legend()
plt.title("Golden Cross Strategy")
plt.show()
```

🧠 Interview Answer:

> A Golden Cross occurs when a short-term moving average crosses above a long-term moving average, indicating a potential bullish trend reversal.

2️⃣ Sharpe Ratio

✅ What is Sharpe Ratio?

Sharpe Ratio measures **risk-adjusted return**.

Formula:

$$\text{Sharpe} = \frac{\text{Return} - \text{RiskFreeRate}}{\text{VolatilitySharpe}}$$

It tells:

👉 How much return you earn per unit of risk.

📊 Code

```
import numpy as np

df['Daily Return'] = df['Close'].pct_change()

mean_return = df['Daily Return'].mean()
volatility = df['Daily Return'].std()

sharpe_ratio = (mean_return / volatility) * np.sqrt(252)
print("Sharpe Ratio:", sharpe_ratio)
```

(252 = trading days per year)

🧠 **Interview Answer:**

> Sharpe Ratio evaluates investment performance by adjusting returns for risk. A higher Sharpe Ratio indicates better risk-adjusted performance.

3️⃣ Correlation Between Two Stocks

Correlation shows how two stocks move together.
Range:

- +1 → move together
- 0 → no relation
- -1 → move opposite

📊 Example with Apple & Tesla

Using:

- Apple Inc.
- Tesla, Inc.

```
import yfinance as yf

aapl = yf.download("AAPL")['Close']
tsla = yf.download("TSLA")['Close']

data = pd.concat([aapl, tsla], axis=1)
data.columns = ['AAPL','TSLA']

correlation = data.pct_change().corr()

print(correlation)
```

🧠 **Interview Answer:**

> Correlation measures how two stocks move relative to each other. It helps in diversification. Low or negative correlation reduces portfolio risk.

4️⃣ Risk vs Return Analysis

📌 **Return**

```
annual_return = df['Daily Return'].mean() * 252
```

📌 **Risk (Volatility)**

```
annual_volatility = df['Daily Return'].std() * np.sqrt(252)
```

📊 Risk-Return Scatter Plot

```
plt.scatter(annual_volatility, annual_return)
plt.xlabel("Risk (Volatility)")
plt.ylabel("Return")
plt.title("Risk vs Return")
plt.show()
```

🧠 **Interpretation:**

- High return + low risk → Ideal
- High risk + low return → Bad investment

5️⃣ Interview Rapid-Fire Questions 🔥

❓ What is Adjusted Close?

It accounts for stock splits and dividends.

❓ Why use 252 in calculations?

There are approximately 252 trading days in a year.

❓ What does high volatility mean?

Larger price fluctuations and higher risk.

❓ Why log returns sometimes preferred?

Log returns are time-additive and better for modeling.

❓ What is diversification?

Investing in assets with low correlation to reduce risk.

❓ What is Beta?

Measures stock sensitivity relative to market.

❓ Difference between technical & fundamental analysis?

- Technical → Charts, patterns, indicators
- Fundamental → Financial statements, earnings, valuation

❓ Difference Between rolling() and resample()

These two are often confused.

🔷 rolling() → Moving Window Calculation

It does calculations over a **fixed number of past observations**.

Example:

```
df['MA20'] = df['Close'].rolling(20).mean()
```

👉 Takes last 20 rows
👉 Computes mean
👉 Moves forward 1 step

📌 ***It does NOT change data frequency.***
It only smooths or calculates statistics.

🔷 **resample() → Change Data Frequency**

It changes time frequency.

Example:

```
df.resample('M').mean()
```

👉 Converts daily data → monthly
👉 Groups all days in a month
👉 Calculates monthly average

It aggregates data.

🔥 **Key Difference**

Feature	rolling()	resample()
Purpose	Moving window stats	Change time frequency
Changes number of rows?	❌ No	✅ Yes
Example	20-day MA	Daily → Monthly
Used for	Smoothing	Aggregation

🧠 **Interview Answer:**

rolling() calculates statistics over a moving window without changing data frequency, while resample() aggregates data into a new time frequency such as daily to monthly or yearly.

❓ **What is Seasonality in Time-Series?**

Definition : Seasonality is a **repeating pattern at regular intervals**.

Examples:

- Ice cream sales higher in summer
- Retail sales higher in December
- Electricity usage higher in winter

In Stock Market?

Stocks may show:

- End-of-year rally
- Monthly patterns
- Quarterly earnings impact

Example Code

```
df['Month'] = df.index.month
monthly_avg = df.groupby('Month')['Close'].mean()
```

This helps detect seasonal pattern.

🔥 **Important**

Seasonality is:

- Predictable
- Repeating
- Time-based pattern

🧠 **Interview Answer:**

Seasonality refers to recurring patterns or fluctuations that occur at regular time intervals in a time-series dataset.

❓ Stationarity (VERY IMPORTANT FOR ML)

This is extremely important for:

- ARIMA
- Time-series forecasting
- Financial modeling

What is Stationarity?

A time-series is **stationary** if:

1. Mean is constant over time
2. Variance is constant
3. No trend or seasonal pattern

Example

❌ Non-stationary:
Stock price of
Apple Inc.

Why?

- It has upward trend
- Mean changes over time

➔ Why ML Needs Stationarity?

Many statistical models assume:

- Data distribution is stable
- Past behavior reflects future behavior

If data has trend:

- Model becomes biased
- Forecast becomes unreliable

1. How To Make Data Stationary?

1️⃣ Differencing

```
df['Diff'] = df['Close'].diff()
```

This removes trend.

2️⃣ Log Transformation

```
import numpy as np
df['Log'] = np.log(df['Close'])
```

Reduces variance instability.

2. How To Check Stationarity?

Using ADF Test:

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df['Close'].dropna())
print("p-value:", result[1])
```

If:

- p-value < 0.05 → stationary
- p-value > 0.05 → non-stationary

🔥 **Visual Explanation**

Non-stationary:

📈 Upward trend

Stationary:

〰 Fluctuates around constant mean

🧠 **Interview-Level Answer:**

A stationary time-series has constant mean, variance, and autocorrelation over time. Most time-series forecasting models assume stationarity, so we often apply transformations like differencing or log transformation to achieve it.

Quick Summary:-

Concept	Meaning
rolling()	Moving window calculation
resample()	Change time frequency
Seasonality	Repeating time-based pattern
Stationarity	Statistical properties constant over time